Tutorial 04: Dockerización y Despliegue en AWS

De Localhost a la Nube: Infraestructura Inmutable

Prof. Nicolás Ramírez Vélez - Arquitectura de Software 2026

1. Objetivo del Laboratorio

En este tutorial, elevaremos la arquitectura de nuestra aplicación "Tienda API" (creada en el Tutorial 03). Dejaremos de ejecutarla manualmente en nuestras máquinas locales para empaquetarla en contenedores **Docker** y desplegarla en una instancia real de **AWS EC2**.

Prerrequisitos:

- Proyecto del Tutorial 03 funcionando (DRF, Serializers y Service Layer).
- Cuenta activa en AWS Academy.
- Docker Desktop instalado localmente.

2. Paso 1: Refactorización de Configuración (The Environment)

Para que la aplicación funcione tanto en su laptop como en Docker o AWS, debemos dejar de "quemar" (hardcode) las credenciales de la base de datos. Usaremos variables de entorno.

2.1. 1.1 Modificar settings.py

Abra su archivo `settings.py` y modifique la configuración de la base de datos para que lea del sistema operativo. Necesitará importar `os` al inicio del archivo.

```
import os

# ... resto del codigo ...

DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.postgresql',
        'NAME': os.environ.get('DB_NAME', 'tienda_db'),
        'USER': os.environ.get('DB_USER', 'postgres'),
        'PASSWORD': os.environ.get('DB_PASSWORD', 'postgres'),
        'HOST': os.environ.get('DB_HOST', 'localhost'), # Importante
    para Docker
        'PORT': os.environ.get('DB_PORT', '5432'),
```

```
    }
}
```

2.2. 1.2 Crear requirements.txt

Asegúrese de listar todas las dependencias. Ejecute en su terminal:

```
pip freeze > requirements.txt
```

Nota: Verifique que 'psycopg2-binary' y 'djangorestframework' estén en la lista.

3. Paso 2: La Receta de la Imagen (Dockerfile)

En la raíz de su proyecto (al mismo nivel que `manage.py`), cree un archivo llamado `Dockerfile` (sin extensión).

```
# 1. Usamos una imagen base oficial de Python ligera
FROM python:3.11-slim

# 2. Evitamos que Python escriba archivos .pyc y buffer de logs
ENV PYTHONDONTWRITEBYTECODE 1
ENV PYTHONUNBUFFERED 1

# 3. Establecemos el directorio de trabajo dentro del contenedor
WORKDIR /app

# 4. Copiamos los requerimientos e instalamos dependencias
COPY requirements.txt /app/
RUN pip install --no-cache-dir -r requirements.txt

# 5. Copiamos el resto del codigo fuente
COPY . /app/

# 6. Comando por defecto al iniciar el contenedor
# Escucha en el puerto 8000 disponible para el mundo (0.0.0.0)
CMD ["python", "manage.py", "runserver", "0.0.0.0:8000"]
```

4. Paso 3: Orquestación Local (Docker Compose)

Para conectar Django con PostgreSQL sin instalar nada en su PC, cree el archivo `docker-compose.yml` en la raíz del proyecto.

```
version: '3.8'

services:
  db:
    image: postgres:15
    volumes:
      - postgres_data:/var/lib/postgresql/data
    environment:
      - POSTGRES_DB=tienda_db
      - POSTGRES_USER=postgres
      - POSTGRES_PASSWORD=secret_password

  web:
```

```
    build: .
    command: python manage.py runserver 0.0.0.0:8000
    volumes:
      - .:/app
    ports:
      - "8000:8000"
    depends_on:
      - db
    environment:
      - DB_NAME=tienda_db
      - DB_USER=postgres
      - DB_PASSWORD=secret_password
      - DB_HOST=db  # Nombre del servicio definido arriba

volumes:
  postgres_data:
```

Prueba Local: Ejecute `docker-compose up --build`. Debería poder acceder a su API en `localhost:8000/api/v1/comprar/`.

5. Paso 4: Despliegue en AWS Academy

Ahora llevaremos esto a la nube. Siga estos pasos quirúrgicos.

5.1. 4.1 Lanzamiento de Instancia EC2

1. Ingrese a AWS Academy LMS y lance el laboratorio.
2. Vaya a la consola de AWS → EC2 → Launch Instance.
3. **Name:** Servidor-Django.
4. **OS Image:** Amazon Linux 2023 AMI (Free tier eligible).
5. **Instance Type:** t2.micro.
6. **Key Pair:** Seleccione `vockey`.
7. **Network Settings:** Marque las casillas .Allow SSH traffic", .Allow HTTP trafficz .Allow HTTPS traffic".
8. Lance la instancia.

5.2. 4.2 Conexión SSH

Haga clic en la instancia creada y presione **Connect**. Use la pestaña .EC2 Instance Connect"para abrir una terminal en el navegador.

5.3. 4.3 Preparación del Quirófano (Instalación de Software)

Ejecute los siguientes comandos en la terminal de AWS para instalar Git y Docker:

```
# Actualizar el sistema
sudo dnf update -y

# Instalar Git y Docker
sudo dnf install git docker -y

# Iniciar el servicio de Docker
sudo service docker start

# Dar permisos al usuario para usar Docker sin sudo
sudo usermod -a -G docker ec2-user

# IMPORTANTE: Cierre la terminal y vuelva a conectar para aplicar
   permisos.
exit
```

5.4. 4.4 Clonación y Despliegue

Vuelva a conectarse a la instancia y traiga su código. *(Nota: Si no tiene su código en GitHub, puede crearlo ahora o usar SCP. Asumiremos GitHub).*

```
# 1. Clonar su repositorio (Use su propia URL)
git clone https://github.com/SU_USUARIO/mi-proyecto-django.git

# 2. Entrar a la carpeta
cd mi-proyecto-django

# 3. Crear el entorno dockerizado en la nube
docker compose up -d --build

# 4. Verificar que esta corriendo
docker ps
```

Nota: Si la versión de Amazon Linux no trae 'docker compose' (v2) por defecto, puede usar 'docker-compose' (v1) o instalar el plugin manualmente.

5.5. 4.5 Exponer al Mundo (Security Groups)

Por defecto, el puerto 8000 está cerrado en AWS.

1. Vaya al panel de EC2 → Instances.
2. Seleccione su instancia → Pestaña "Security".
3. Haga clic en el Security Group (ej: `launch-wizard-1`).
4. **Edit inbound rules** → Add rule.
5. **Type:** Custom TCP → **Port range:** 8000 → **Source:** 0.0.0.0/0 (Anywhere).
6. Save rules.

6. Entrega de Resultados

Para validar el laboratorio, tome una captura de pantalla de su navegador accediendo a: `http://<IP-PUBLICA-EC2>:8000/api/v1/comprar/` Debe verse la respuesta JSON de su API servida desde AWS.